Lab 17– AI for Data Processing: Data cleaning and preprocessing scripts

The objective of this lab is to enable students to understand and apply AI-assisted coding tools for automating and enhancing data preprocessing tasks. Students will:

1. Gain practical experience in cleaning, transforming, and standardizing real-world datasets with issues such as missing values, duplicates, outliers, inconsistent formats, and noisy text.
2. Learn to leverage AI coding assistants to generate preprocessing scripts, while critically evaluating and refining the AI-generated code for accuracy, efficiency, and best practices.
3. Develop the ability to design end-to-end preprocessing pipelines that prepare raw data for downstream machine learning and analytics applications.
4. Build confidence in combining human expertise with AI assistance, ensuring data quality and integrity in diverse domains such as customer feedback, healthcare, and finance

Lab Question 1: Customer Feedback Dataset

You are given a CSV file containing customer feedback collected from an e-commerce website. The dataset includes columns: customer_id, feedback_text, rating, and date. However, the file has many missing values, typos, and inconsistent date formats.

- Task 1: Use an AI-assisted coding tool to generate a script that detects and fills missing rating values with the column's median and standardizes the date column into YYYY-MM-DD format.
- Task 2: Clean the feedback_text column by removing stopwords, correcting common spelling mistakes, and converting text to lowercase using AI suggestions. Compare the AI-generated preprocessing code with your manually written version.

```python
import pandas as pd
import numpy as np
from textblob import TextBlob
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
df_feedback = pd.read_csv("/content/customer_feedback.csv")
print("Original Customer Feedback Data:\n", df_feedback.head(),"\n")
df_feedback['rating'].fillna(df_feedback['rating'].median(), inplace=True)
df_feedback['date'] = pd.to_datetime(df_feedback['date'], errors='coerce').dt.strf

print("After Handling Missing Ratings & Date Format:\n", df_feedback.head(),"\n")
```

```
Original Customer Feedback Data:
    customer_id                   feedback_text  rating        date
0            1         Great product! Loved it     5.0  2025/10/01
1            2  bad quallity and late delivary     2.0  10-02-2025
2            3                             NaN     NaN  Oct 3 2025
3            4  AVERAGE service but price high     3.0  2025.10.04
4            5      excellent!! will buy again     5.0  2025-10-05

After Handling Missing Ratings & Date Format:
    customer_id                   feedback_text  rating        date
0            1         Great product! Loved it     5.0  2025-10-01
1            2  bad quallity and late delivary     2.0         NaN
2            3                             NaN     3.0         NaN
3            4  AVERAGE service but price high     3.0         NaN
4            5      excellent!! will buy again     5.0         NaN
```

```python
import pandas as pd
import numpy as np
from textblob import TextBlob
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

# --- Load data ---
df_feedback = pd.read_csv("/content/customer_feedback.csv")

# --- Handle missing ratings safely ---
df_feedback['rating'] = df_feedback['rating'].fillna(df_feedback['rating'].median())

# --- Define cleaning function ---
def clean_text_manual(text):
    if pd.isnull(text):       # handles NaN safely
        return ""
    text = str(text).lower()  # convert safely to lowercase
    words = [w for w in text.split() if w not in stopwords.words('english')]
    corrected = " ".join([str(TextBlob(w).correct()) for w in words])
    return corrected

# --- Apply cleaning function ---
df_feedback['cleaned_text_manual'] = df_feedback['feedback_text'].apply(clean_text_manual)
print(df_feedback.head())
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
   customer_id                 feedback_text  rating        date  \
0            1        Great product! Loved it     5.0  2025/10/01

  cleaned_text_manual
0       great product!
1  bad quality delivery
2
3  average service price high
4            excellent!!
```

Lab Question 2: Medical Records Dataset

A hospital provides you with a dataset of anonymized medical records containing attributes like patient_id, age, gender, blood_pressure, and

cholesterol. Some columns include outliers and inconsistent categorical labels (e.g., Male, M, male).

• Task 1: Write a script (with AI assistance) to detect and handle outliers in the blood_pressure column using statistical methods (e.g., IQR or z-score).

• Task 2: Standardize categorical values in the gender column and encode them into numeric form. Let the AI-assisted coding tool propose the preprocessing pipeline, then refine the pipeline

manually based on your understanding

```
df_med = pd.read_csv("/content/medical_records.csv")
print("🟢 Original Medical Records Data:\n", df_med.head(),"\n")
Q1 = df_med['blood_pressure'].quantile(0.25)
Q3 = df_med['blood_pressure'].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5*IQR, Q3 + 1.5*IQR
df_med_outliers_removed = df_med[(df_med['blood_pressure'] >= lower) & (df_med['blood_pressure'] <= upper)]
print("✅ After removing outliers:\n", df_med_outliers_removed, "\n")
```

```
🟢 Original Medical Records Data:
   patient_id  age  gender  blood_pressure  cholesterol
0         101   58       f             120          180
1         102   71       M             200          220
2         103   48       M             110          150
3         104   34  female              90          170
4         105   62       F             300          310

✅ After removing outliers:
   patient_id  age  gender  blood_pressure  cholesterol
0         101   58       f             120          180
1         102   71       M             200          220
2         103   48       M             110          150
3         104   34  female              90          170
4         105   62       F             300          310
5         106   27       M             130          200
6         107   40  Female              85          140
7         108   58  female             250          290
8         109   77    male             140          230
9         110   38       F              95          160
```

```
import pandas as pd
import numpy as np

df_med = pd.read_csv("/content/medical_records.csv")
print("🟢 Original Medical Records Data:\n", df_med.head(), "\n")

# --- Task 2: Standardize Gender Labels & Encode ---
def standardize_gender(value):
    if pd.isnull(value):
        return np.nan
    value = str(value).strip().lower()
    if value in ['m','male']:
        return 'Male'
    elif value in ['f','female']:
        return 'Female'
    else:
        return 'Other'

# Apply the standardization function
df_med['gender_standardized'] = df_med['gender'].apply(standardize_gender)

# Encode standardized genders to numeric
df_med['gender_encoded'] = df_med['gender_standardized'].map({'Male':1, 'Female':0, 'Other':2})

print("✅ After gender standardization & encoding:\n",
      df_med[['gender','gender_standardized','gender_encoded']], "\n")
```

```
🟢 Original Medical Records Data:
   patient_id  age  gender  blood_pressure  cholesterol
0         101   58       f             120          180
1         102   71       M             200          220
2         103   48       M             110          150
3         104   34  female              90          170
4         105   62       F             300          310

✅ After gender standardization & encoding:
   gender gender_standardized  gender_encoded
0       f              Female               0
1       M                Male               1
2       M                Male               1
3  female              Female               0
4       F              Female               0
5       M                Male               1
6  Female              Female               0
7  female              Female               0
8    male                Male               1
9       F              Female               0
```

Lab Question 3: Financial Transactions Dataset

A bank gives you transaction data with columns: transaction_id, amount, currency, timestamp, and merchant. The dataset contains multiple issues: different currency units (USD, INR, EUR), timestamps in various time zones, and duplicated rows.

• Task 1: Use AI-assisted coding to write a script that removes duplicate transactions and converts all amount values into a single currency (e.g., USD) using a provided conversion dictionary.

• Task 2: Normalize the timestamp column into UTC format and create a new column transaction_hour for downstream time-series analysis. Compare the AI's preprocessing code against your own optimized version.

```python
df_fin = pd.read_csv("/content/financial_transactions.csv")
print("🟢 Original Financial Transactions Data:\n", df_fin.head(), "\n")
conversion = {'USD':1, 'INR':0.012, 'EUR':1.08}
df_fin.drop_duplicates(inplace=True)
df_fin['amount_usd'] = df_fin.apply(lambda x: x['amount'] * conversion.get(x['currency'],1), axis=1)
print("✅ After currency conversion & duplicate removal:\n", df_fin.head(), "\n")
conversion = {'USD':1, 'INR':0.012, 'EUR':1.08}
df_fin.drop_duplicates(inplace=True)
df_fin['amount_usd'] = df_fin.apply(lambda x: x['amount'] * conversion.get(x['currency'],1), axis=1)
print("✅ After currency conversion & duplicate removal:\n", df_fin.head(), "\n")
```

```
🟢 Original Financial Transactions Data:
   transaction_id  amount currency            timestamp merchant
0               1  284.47      EUR  2025-10-11 15:00:00   Amazon
1               2  381.95      EUR  2025-10-10 14:00:00  Walmart
2               3  403.43      USD  2025-10-11 23:00:00   Amazon
3               4  982.29      USD  2025-10-12 00:00:00   Amazon
4               4  143.78      USD  2025-10-10 16:00:00   Amazon

✅ After currency conversion & duplicate removal:
   transaction_id  amount currency            timestamp merchant  amount_usd
0               1  284.47      EUR  2025-10-11 15:00:00   Amazon    307.2276
1               2  381.95      EUR  2025-10-10 14:00:00  Walmart    412.5060
2               3  403.43      USD  2025-10-11 23:00:00   Amazon    403.4300
3               4  982.29      USD  2025-10-12 00:00:00   Amazon    982.2900
4               4  143.78      USD  2025-10-10 16:00:00   Amazon    143.7800
```


```
✅ After currency conversion & duplicate removal:
   transaction_id  amount currency            timestamp merchant  amount_usd
0               1  284.47      EUR  2025-10-11 15:00:00   Amazon    307.2276
1               2  381.95      EUR  2025-10-10 14:00:00  Walmart    412.5060
2               3  403.43      USD  2025-10-11 23:00:00   Amazon    403.4300
3               4  982.29      USD  2025-10-12 00:00:00   Amazon    982.2900
4               4  143.78      USD  2025-10-10 16:00:00   Amazon    143.7800

✅ After currency conversion & duplicate removal:
   transaction_id  amount currency            timestamp merchant  amount_usd
0               1  284.47      EUR  2025-10-11 15:00:00   Amazon    307.2276
1               2  381.95      EUR  2025-10-10 14:00:00  Walmart    412.5060
2               3  403.43      USD  2025-10-11 23:00:00   Amazon    403.4300
3               4  982.29      USD  2025-10-12 00:00:00   Amazon    982.2900
4               4  143.78      USD  2025-10-10 16:00:00   Amazon    143.7800
```

```python
# Load the dataset
df_fin = pd.read_csv("/content/financial_transactions.csv")
print("🟢 Original Financial Transactions Data:\n", df_fin.head(), "\n")

# --- Task 2: Normalize Timestamp & Add Hour ---
# Convert 'timestamp' to datetime object
df_fin['timestamp'] = pd.to_datetime(df_fin['timestamp'], errors='coerce', utc=True)

# Create a new column for transaction hour
df_fin['transaction_hour'] = df_fin['timestamp'].dt.hour

print("✅ After timestamp normalization & hour extraction:\n",
      df_fin[['timestamp','transaction_hour']].head(), "\n")
```

```
🟢 Original Financial Transactions Data:
   transaction_id  amount currency            timestamp merchant
0               1  284.47      EUR  2025-10-11 15:00:00   Amazon
1               2  381.95      EUR  2025-10-10 14:00:00  Walmart
2               3  403.43      USD  2025-10-11 23:00:00   Amazon
3               4  982.29      USD  2025-10-12 00:00:00   Amazon
4               4  143.78      USD  2025-10-10 16:00:00   Amazon

✅ After timestamp normalization & hour extraction:
                  timestamp  transaction_hour
0 2025-10-11 15:00:00+00:00              15.0
1 2025-10-10 14:00:00+00:00              14.0
2 2025-10-11 23:00:00+00:00              23.0
3 2025-10-12 00:00:00+00:00               0.0
4 2025-10-10 16:00:00+00:00              16.0
```

Summary:the ai generated code was fast but not too accurate.where as manual code takes a lot od time to code but even error are prone in manually eritten code also.hence we need to double check both the codes but comparatively ai version is clean,neat and reduces time.But in cases of csv file creation a lot of time is cusumed if we don’t use the help of ai to generate dataset but if we do manually it will be error free